



# Προστασία και Ασφάλεια Υπολογιστικών και Τηλεπικοινωνιακών Συστημάτων — 3η εργασία προς παράδοση

## Στοιχεία φοιτητή

Ονοματεπώνυμο: Φωτόπουλος Γεώργιος Κωνσταντίνος

A.M.: 1117202200234



Ολόκληρο το παρόν PDF το έχω κάνει με χρήση του **Notion**.

## Κώδικας

### CSS

**style.css**

```
/**  
 * css/style.css  
 *  
 * Full Name: George Constantine Fotopoulos
```

```
* R.N.: 1117202200234
*/

body {
  margin: 16px;
  font-size: 16px;
  font-family: Arial, sans-serif;
}

form {
  width: 360px;
}

label {
  display: block;
  margin-top: 12px;
  margin-bottom: 4px;
}

input, button {
  font-size: inherit;
}

input[type="text"], input[type="password"] {
  width: 100%;
  max-width: 260px;
}

button {
  margin-top: 12px;
  margin-right: 8px;
}

.message-area {
  margin-bottom: 16px;
}
```

```

.message-area p {
    margin-top: 0;
    margin-bottom: 8px;
}

.result-item {
    margin-bottom: 20px;
}

.product-table {
    border-collapse: collapse;
    margin-top: 16px;
}

.product-table th, .product-table td {
    border: 1px solid black;
    padding: 6px 10px;
    text-align: left;
}

```

## JavaScript

### login\_validation.js

```

/**
 * js/login_validation.js
 *
 * Full Name: George Constantine Fotopoulos
 * R.N.: 1117202200234
 */

function ValidateLoginForm() {
    const username_input = document.getElementById("username");
    const password_input = document.getElementById("password");
}

```

```

    if (username_input.value.trim() === "" || password_input.value.trim() === "") {
        alert("Please fill in both the username and password fields.");
        return false;
    }

    return true;
}

```

## PHP

### common.php

```

<?php

use PDO\Pgsql;

/**
 * common.php
 *
 * Full Name: George Constantine Fotopoulos
 * R.N.: 1117202200234
 */

# --- Constants ---

define ("APP_TITLE", "PostgreSQL Injection");

define("DATABASE_HOST", "localhost");
define("DATABASE_PORT", "5432");
define("DATABASE_NAME", "sql_injection");
define("DATABASE_USERNAME", "postgres");
define("DATABASE_PASSWORD", "postgres");

```

```

define("TABLE_USERS",    "Users");
define("TABLE_PRODUCT",  "Product");

# --- Helper Functions ---

function EscapeHtml(string $value): string {
    return htmlspecialchars($value, ENT_QUOTES, "UTF-8");
}

function ConnectToDatabase(): \PgSql\Connection {
    $connection_string =
        "host="      . DATABASE_HOST .
        " port="     . DATABASE_PORT .
        " dbname="   . DATABASE_NAME .
        " user="     . DATABASE_USERNAME .
        " password=" . DATABASE_PASSWORD;

    $connection = pg_connect($connection_string);

    if ($connection === false) {
        die("[Error] Failed to connect to the PostgreSQL database.");
    }

    return $connection;
}
?>

```

## index.php

```

<?php
/**
 * index.php
 *
 * Full Name: George Constantine Fotopoulos

```

```

* R.N.: 1117202200234
*/

require_once("common.php");
?>

<!DOCTYPE html>
<html lang="en">

<!-- Head -->

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Assignment 3 - <?php echo EscapeHtml(APP_TITLE) ?>
</title>

  <link rel="stylesheet" href="css/style.css">

  <script src="js/login_validation.js"></script>
</head>

<!-- Body -->

<body>
  <h1><?php echo EscapeHtml(APP_TITLE) ?></h1>

  <p>
    This application demonstrates how an SQL injection attack can bypass
    vulnerable authentication systems and gain unauthorised access.
  </p>

  <form method="post" action="vulnerable_login.php" onsubmi

```

```

t="return ValidateLoginForm();">
    <label for="username">Username</label>
    <input type="text" id="username" name="username">

    <label for="password">Password</label>
    <input type="password" id="password" name="password">

    <br>

    <button type="submit" formaction="vulnerable_login.ph
p">
        Vulnerable Login
    </button>

    <button type="submit" formaction="multi_statement_logi
n.php">
        Multi-Statement Demo
    </button>

    <button type="submit" formaction="safe_login.php">
        Safe Login
    </button>
</form>
</body>
</html>

```

### vulnerable\_login.php

```

<?php
/**
 * vulnerable_login.php
 *
 * Full Name: George Constantine Fotopoulos
 * R.N.: 1117202200234
 */

```

```

require_once("common.php");

# --- Request Handling Logic ---

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    die("[Error] Invalid request method.");
}

$username = trim($_POST["username"] ?? "");
$password = trim($_POST["password"] ?? "");

if ($username === "" || $password === "") {
    die("[Error] Both the username and password are required.");
}

$connection = ConnectToDatabase();

/*
 * This query is intentionally vulnerable to SQL injection.
 *
 * Reason:
 * - The user input is directly concatenated into the SQL command.
 *
 * Fix:
 * - Structure it as a **parameterised query**; with '$1' and '$2',
 *   instead of '$username' and '$password'.
 *
 * This makes it so that SQL expressions like "' OR '1'='1" is not
 * seen as SQL logic, but rather as a simple password's value.
 */

```



```

$query =
    "SELECT username " .
    "FROM " . TABLE_USERS . " " .
    "WHERE username = '" . $username . "' " .
    "AND password = '" . $password . "'";

$result = pg_query($connection, $query);

if ($result === false) {
    die("[Error] The authentication query failed.");
}

$user = pg_fetch_assoc($result);

$products_result = false;

if ($user !== false) {
    $products_query =
        "SELECT product_id, product_name, price " .
        "FROM " . TABLE_PRODUCT . " " .
        "ORDER BY product_id";

    $products_result = pg_query($connection, $products_query);

    if ($products_result === false) {
        die("[Error] The products query failed.");
    }
}
?>

<!DOCTYPE html>
<html lang="en">

<!-- Head -->

```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Assignment 3 - <?php echo EscapeHtml(APP_TITLE) ?>
</title>

  <link rel="stylesheet" href="css/style.css">
</head>

<!-- Body -->

<body>
  <h1><?php echo EscapeHtml(APP_TITLE) ?></h1>

  <?php if ($user === false): ?>
    <div class="message-area">
      <p>Access denied.</p>
      <p>You do not have permission to access the product data.</p>
    </div>
  <?php else: ?>
    <div class="message-area">
      <p>Access granted.</p>
      <p>Authenticated user: <?php echo EscapeHtml($user["username"]); ?></p>
    </div>

    <table class="product-table">
      <thead>
        <tr>
          <th>Product ID</th>
          <th>Product Name</th>
          <th>Price</th>
        </tr>
      </thead>

```

```

        <tbody>
            <?php while (($product = pg_fetch_assoc($products_result)) !== false): ?>
                <tr>
                    <td><?php echo EscapeHtml($product["product_id"]); ?></td>
                    <td><?php echo EscapeHtml($product["product_name"]); ?></td>
                    <td><?php echo EscapeHtml($product["price"]); ?></td>
                </tr>
            <?php endwhile; ?>
        </tbody>
    </table>
    <?php endif; ?>

    <p>
        <a href="index.php">Back to login form</a>
    </p>
</body>
</html>

```

### multi-statement-login.php

```

<?php
/**
 * multi_statement_login.php
 *
 * Full Name: George Constantine Fotopoulos
 * R.N.: 1117202200234
 */

require_once("common.php");

```

```

# --- Request Handling Logic ---

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    die("[Error] Invalid request method.");
}

$username = trim($_POST["username"] ?? "");
$password = trim($_POST["password"] ?? "");

if ($username === "" || $password === "") {
    die("[Error] Both the username and password are required.");
}

$connection = ConnectToDatabase();

/*
 * This query is intentionally vulnerable to SQL injection.
 *
 * Reason:
 * - The user input is directly concatenated into the SQL command.
 *
 * Fix:
 * - Structure it as a **parameterised query**; with '$1' and '$2',
 *   instead of '$username' and '$password'.
 *
 * This makes it so that SQL expressions like "' OR '1'='1" is not
 * seen as SQL logic, but rather as a simple password's value.
 */
$query =
    "SELECT username " .
    "FROM " . TABLE_USERS . " " .

```

```

        "WHERE username = '" . $username . "' " .
        "AND password = '" . $password . "'";

$send_result = pg_send_query($connection, $query);

if ($send_result === false) {
    die("[Error] The multi-statement query could not be sent.");
}

$authentication_result = pg_get_result($connection);

if ($authentication_result === false) {
    die("[Error] The authentication query could not be read.");
}

$user = pg_fetch_assoc($authentication_result);

// Consume any additional results produced by the extra injected SQL statements.
while (pg_get_result($connection) !== false) {}

$products_result = false;

if ($user !== false) {
    $products_query =
        "SELECT product_id, product_name, price " .
        "FROM " . TABLE_PRODUCT . " " .
        "ORDER BY product_id";

    $products_result = pg_query($connection, $products_query);

    if ($products_result === false) {
        die("[Error] The products query failed.");
    }
}

```

```

    }
}
?>

<!DOCTYPE html>
<html lang="en">

<!-- Head -->

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Assignment 3 - <?php echo EscapeHtml(APP_TITLE) ?>
</title>

    <link rel="stylesheet" href="css/style.css">
</head>

<!-- Body -->

<body>
    <h1><?php echo EscapeHtml(APP_TITLE) ?></h1>

    <?php if ($user === false): ?>
        <div class="message-area">
            <p>Access denied.</p>
            <p>You do not have permission to access the product data.</p>
        </div>
    <?php else: ?>
        <div class="message-area">
            <p>Access granted.</p>
            <p>Authenticated user: <?php echo EscapeHtml($user["username"]); ?></p>
        </div>
    </body>
</html>

```

```

<table class="product-table">
  <thead>
    <tr>
      <th>Product ID</th>
      <th>Product Name</th>
      <th>Price</th>
    </tr>
  </thead>

  <tbody>
    <?php while (($product = pg_fetch_assoc($products_r
esult)) !== false): ?>
      <tr>
        <td><?php echo EscapeHtml($product["product_i
d"]); ?></td>
        <td><?php echo EscapeHtml($product["product_nam
e"]); ?></td>
        <td><?php echo EscapeHtml($product["price"]); ?
></td>
      </tr>
    <?php endwhile; ?>
  </tbody>
</table>
<?php endif; ?>

<p>
  <a href="index.php">Back to login form</a>
</p>
</body>
</html>

```

**safe\_login.php**

```

<?php
/**
 * safe_login.php
 *
 * Full Name: George Constantine Fotopoulos
 * R.N.: 1117202200234
 */

require_once("common.php");

# --- Request Handling Logic ---

if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    die("[Error] Invalid request method.");
}

$username = trim($_POST["username"] ?? "");
$password = trim($_POST["password"] ?? "");

if ($username === "" || $password === "") {
    die("[Error] Both the username and password are required.");
}

$connection = ConnectToDatabase();

/**
 * This query is protected against SQL injection.
 *
 * The $1 and $2 placeholders are replaced safely by PostgreSQL.
 * Therefore, the user input is treated as data, rather than SQL code.
 */
$query =

```



```

        "SELECT username " .
        "FROM " . TABLE_USERS . " " .
        "WHERE username = $1 " .
        "AND password = $2";

$result = pg_query_params($connection, $query, [$username,
$password]);

if ($result === false) {
    die("[Error] The authentication query failed.");
}

$user = pg_fetch_assoc($result);

$products_result = false;

if ($user !== false) {
    $products_query =
        "SELECT product_id, product_name, price " .
        "FROM " . TABLE_PRODUCT . " " .
        "ORDER BY product_id";

    $products_result = pg_query($connection, $products_query);

    if ($products_result === false) {
        die("[Error] The products query failed.");
    }
}
?>

<!DOCTYPE html>
<html lang="en">

<!-- Head -->

```

```

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Assignment 3 - <?php echo EscapeHtml(APP_TITLE) ?>
</title>

  <link rel="stylesheet" href="css/style.css">
</head>

<!-- Body -->

<body>
  <h1><?php echo EscapeHtml(APP_TITLE) ?></h1>

  <?php if ($user === false): ?>
    <div class="message-area">
      <p>Access denied.</p>
      <p>You do not have permission to access the product data.</p>
    </div>
  <?php else: ?>
    <div class="message-area">
      <p>Access granted.</p>
      <p>Authenticated user: <?php echo EscapeHtml($user["username"]); ?></p>
    </div>

    <table class="product-table">
      <thead>
        <tr>
          <th>Product ID</th>
          <th>Product Name</th>
          <th>Price</th>
        </tr>
      </thead>

```

```

        <tbody>
            <?php while (($product = pg_fetch_assoc($products_r
esult)) !== false): ?>
                <tr>
                    <td><?php echo EscapeHtml($product["product_i
d"]); ?></td>
                    <td><?php echo EscapeHtml($product["product_nam
e"]); ?></td>
                    <td><?php echo EscapeHtml($product["price"]); ?
></td>
                </tr>
            <?php endwhile; ?>
        </tbody>
    </table>
    <?php endif; ?>

    <p>
        <a href="index.php">Back to login form</a>
    </p>
</body>
</html>

```

## SQL

### schema.sql

```

-- sql/schema.sql
--
-- Full Name: George Constantine Fotopoulos
-- R.N.: 1117202200234
--
-- 1. Setup the tables
--

```

```
DROP TABLE IF EXISTS Product;  
DROP TABLE IF EXISTS Users;
```

```
--  
--  
--
```

2. Create the tables

```
CREATE TABLE Users (  
    username VARCHAR(30) PRIMARY KEY,  
    password VARCHAR(255) NOT NULL,  
);  
  
CREATE TABLE Product (  
    product_id INTEGER GENERATED ALWAYS AS IDENTITY PR  
IMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    price NUMERIC(10, 2) NOT NULL  
);
```

```
--  
--  
--
```

3. Insert the users and

```
INSERT INTO Users (username, password)  
VALUES  
    ("mort", "pizza"),  
    ("bob", "bob123"),  
    ("admin", "admin");  
  
INSERT INTO Product (product_name, price)  
VALUES  
    ("Laptop", 2899.99),  
    ("Mouse", 59.99),
```

```
("Keyboard", 119.99),  
("Monitor", 379.99);
```

### reset\_products.sql

```
-- sql/reset_products.sql  
--  
-- Full Name: George Constantine Fotopoulos  
-- R.N.: 1117202200234  
  
-- Remove all product rows and reset the identity counter back to 1.  
TRUNCATE TABLE Product RESTART IDENTITY;  
  
INSERT INTO Product (product_name, price)  
VALUES  
    ("Laptop", 2899.99),  
    ("Mouse", 59.99),  
    ("Keyboard", 119.99),  
    ("Monitor", 379.99);
```

## Αναφορά

### Σκοπός της εργασίας

Στην παρούσα εργασία υλοποιείται ένα παράδειγμα παραβίασης ελέγχου πρόσβασης με χρήση SQL injection (σε PostgreSQL).

Αρχικά, ο χρήστης συμπληρώνει τα πεδία `username` και `password` μέσω της HTML φόρμας.

Αν τα στοιχεία αυτά υπάρχουν στον πίνακα `Users`, τότε εμφανίζονται τα δεδομένα του πίνακα `Product`.

### Κανονική λειτουργία

Τα παρακάτω έγκυρα στοιχεία χρήστη:

```
Username: mort  
Password: pizza
```

μας δίνει το αποτέλεσμα:

```
Access granted.
```

και εμφανίζεται ο πίνακας με τα προϊόντα.

Για μη έγκυρα στοιχεία χρήστη:

```
Username: mort  
Password: ablalabla
```

μας δίνει το αποτέλεσμα:

```
Access denied.
```

## SQL Injection

### Υπόθεση 1

Ο κακόβουλος χρήστης γνωρίζει το username ενός έγκυρου χρήστη, αλλά όχι τον συνθηματικό του· για παράδειγμα:

```
Username: alice  
Password: ' OR '1'='1
```

όπου η ευάλωτη SQL εντολή γίνεται:

```
SELECT username  
FROM Users
```

```
WHERE username = 'alice'
AND password = '' OR '1'='1'
```

δηλαδή, η συνθήκη `'1'='1'` είναι πάντα αληθής.

Άρα, το αποτέλεσμα του ελέγχου ταυτότητας επιστρέφει την εγγραφή από τον πίνακα `Users` που αντιστοιχεί στο δοθέν όνομα χρήστη, και η εφαρμογή θεωρεί ότι ο χρήστης έχει δικαίωμα πρόσβασης.

Με αυτόν τον τρόπο εμφανίζονται τα δεδομένα του πίνακα `Product`, χωρίς ο χρήστης να γνωρίζει το πραγματικό συνθηματικό.

## Υπόθεση 2

Ο κακόβουλος χρήστης δε γνωρίζει κανένα έγκυρο username, για παράδειγμα:

```
Username: ' OR '1'='1' --
Password: anything
```

όπου η ευάλωτη SQL εντολή γίνεται:

```
SELECT username
FROM Users
WHERE username = '' OR '1'='1' -- '
AND password = 'anything'
```

δηλαδή το `--` στην PostgreSQL ξεκινά ένα σχόλιο μέχρι το τέλος της γραμμής.

Άρα, το υπόλοιπο κομμάτι της SQL εντολής αγνοείται:

```
AND password = 'anything'
```

κι άρα, ουσιαστικά η συνθήκη γίνεται:

```
username = '' OR '1'='1'
```

για τον ίδιο λόγο ότι το `'1'='1'` είναι πάντα αληθές, το query επιστρέφει τουλάχιστον μία γραμμή από τον πίνακα `Users`.

Στον PHP κώδικα, η εντολή `pg_fetch_assoc($result)` δεν επιστρέφει `false`, οπότε η εφαρμογή θεωρεί ότι ο χρήστης έχει δικαίωμα πρόσβασης.

Έτσι εμφανίζονται τα δεδομένα του πίνακα `Product`, χωρίς ο χρήστης να γνωρίζει ούτε το όνομα χρήστη ούτε το συνθηματικό.

## Multi-statement SQL injection

Στο αρχείο `multi_statement_login.php`, η εφαρμογή χρησιμοποιεί την `pg_send_query()`.

Σ' αυτήν την περίπτωση, μπορεί να εκτελεστεί ένα query string που να περιέχει περισσότερες από μία SQL εντολές.

Παράδειγμα εισόδου:

```
Username: alice'; INSERT INTO Product(product_name, price) VALUES("Injected Product", 0.01);
Password: anything
```

δηλαδή, η SQL εντολή που δημιουργείται περιέχει:

```
SELECT username
FROM users
WHERE username = 'alice';

INSERT INTO Product(product_name, price)
VALUES ("Injected Product", 0.01);
```

Το αποτέλεσμα είναι ότι ο χρήστης παρακάμπτει τον έλεγχο πρόσβασης, και ταυτόχρονα εισάγει ένα νέο προϊόν στον πίνακα `Product`.

Αυτό δείχνει ότι η δυνατότητα εκτέλεσης πολλαπλών SQL εντολών κάνει την εφαρμογή πιο επικίνδυνη, όταν ο κώδικας είναι ευάλωτος σε SQL injection.

## Ευπαθές σημείο κώδικα & Ασφαλής υλοποίηση

Η ευπάθεια υπάρχει για τον λόγο ότι ο PHP κώδικας δημιουργεί την SQL εντολή με απευθείας συνένωση των δεδομένων εισόδου:



```
$query =  
"SELECT username "  
"FROM " . TABLE_USERS . " "  
"WHERE username = '" . $username . "' "  
"AND password = '" . $password . "'";
```

Το πρόβλημα είναι ότι τα δεδομένα του χρήστη αντιμετωπίζονται ως μέρος μιας SQL εντολής.

Η ασφαλής λύση, είναι η χρήση ενός parameterised query, όπως ακριβώς χρησιμοποιείται στο αρχείο `safe_login.php`:

```
$query =  
"SELECT username "  
"FROM " . TABLE_USERS . " "  
"WHERE username = $1 "  
"AND password = $2";  
  
$result = pg_query_params($connection, $query, [$username, $password]);
```

Με αυτόν τον τρόπο, οι τιμές που δίνει ο χρήστης δεν εκτελούνται ως SQL κώδικας.

Για παράδειγμα, το παρακάτω:

```
' OR '1'='1
```

αντιμετωπίζεται ως μια απλή τιμή του password, κι όχι ως SQL λογική.

## Extra

### Ελεγχος ορθότητας των αρχείων

Ο έλεγχος ορθότητας των αρχείων για συντακτικά σφάλματα μπορεί να γίνει με χρήση των παρακάτω εντολών:

```
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\common.php
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\index.php
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\vulnerable_login.php
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\multi-statement-login.php
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\safe_login.php
```